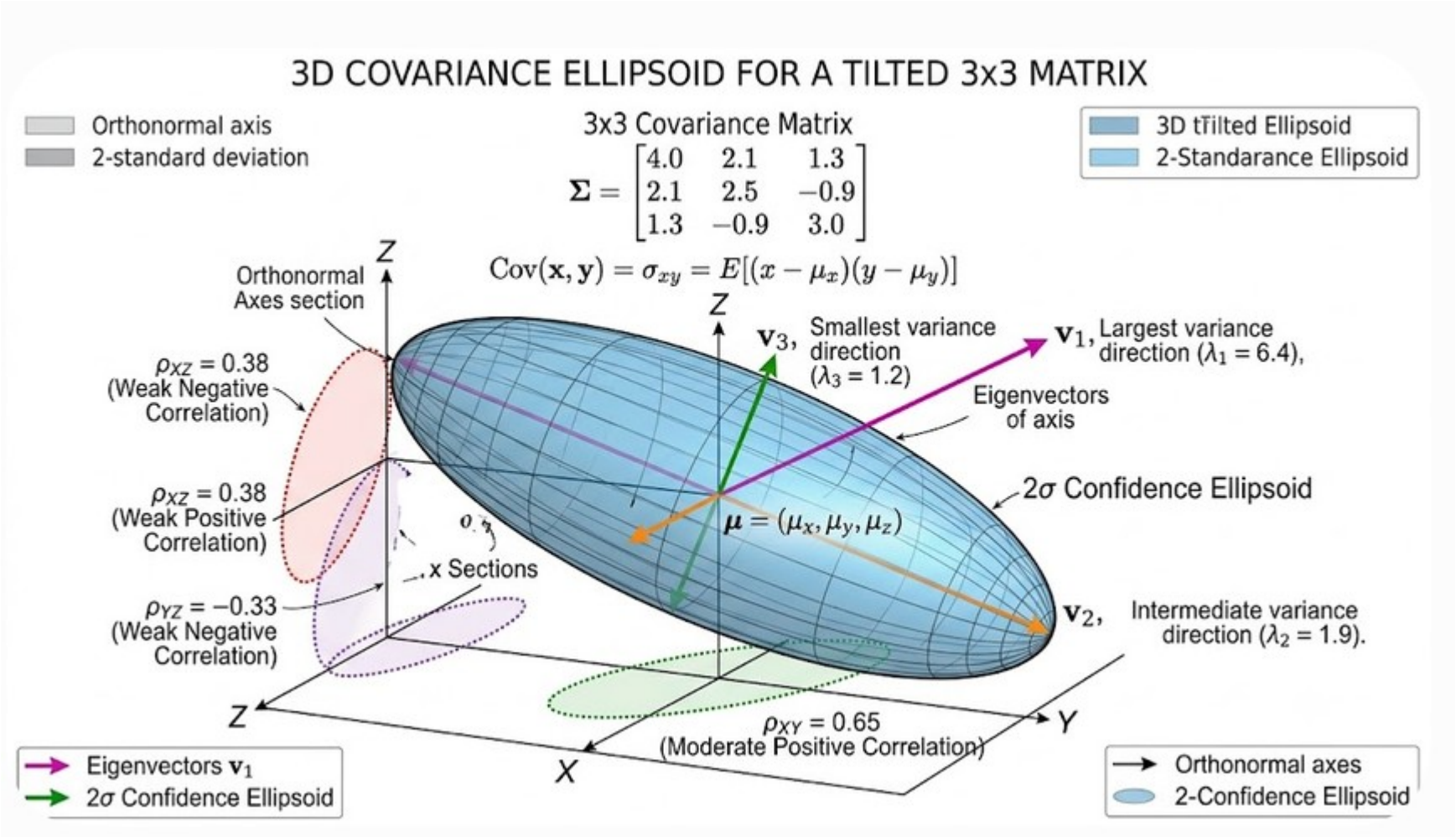
Demonstration of Variance-Covariance Principles

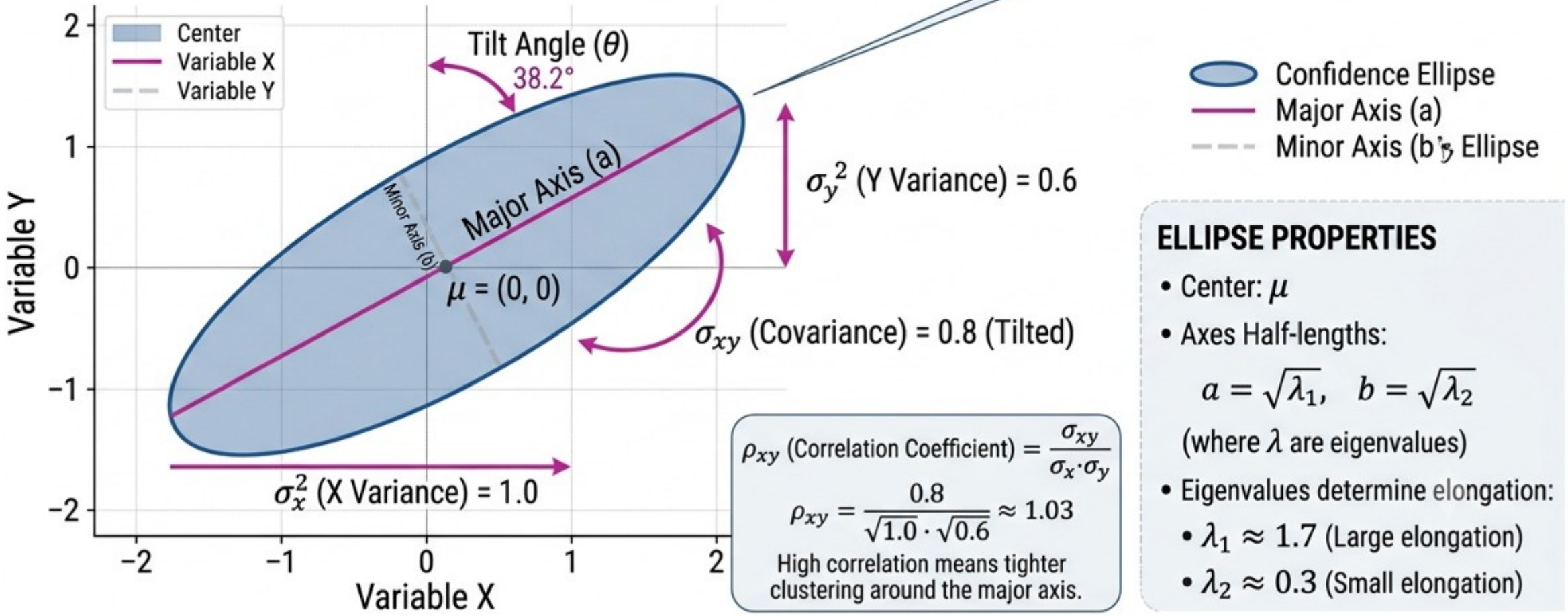


Demonstration of Variance-Covariance Principles

VISUALIZING THE COVARIANCE MATRIX IN 2D: VARIABLE CORRELATION & ELLIPSOID GEOMETRY

$$\Sigma = \begin{vmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{yx} & \sigma_y^2 \end{vmatrix} \quad \text{EXAMPLE } \Sigma = \begin{vmatrix} 1.0 & 0.8 \\ 0.8 & 0.6 \end{vmatrix}$$

Tilt indicates non-zero covariance (correlation).
Larger values mean more elongation.



Confidence Ellipse
Major Axis (a)
Minor Axis (b) Ellipse

$$\rho_{xy} \text{ (Correlation Coefficient)} = \frac{\sigma_{xy}}{\sigma_x \cdot \sigma_y}$$

$$\rho_{xy} = \frac{0.8}{\sqrt{1.0} \cdot \sqrt{0.6}} \approx 1.03$$

High correlation means tighter clustering around the major axis.

ELLIPSE PROPERTIES

- Center: μ
- Axes Half-lengths:

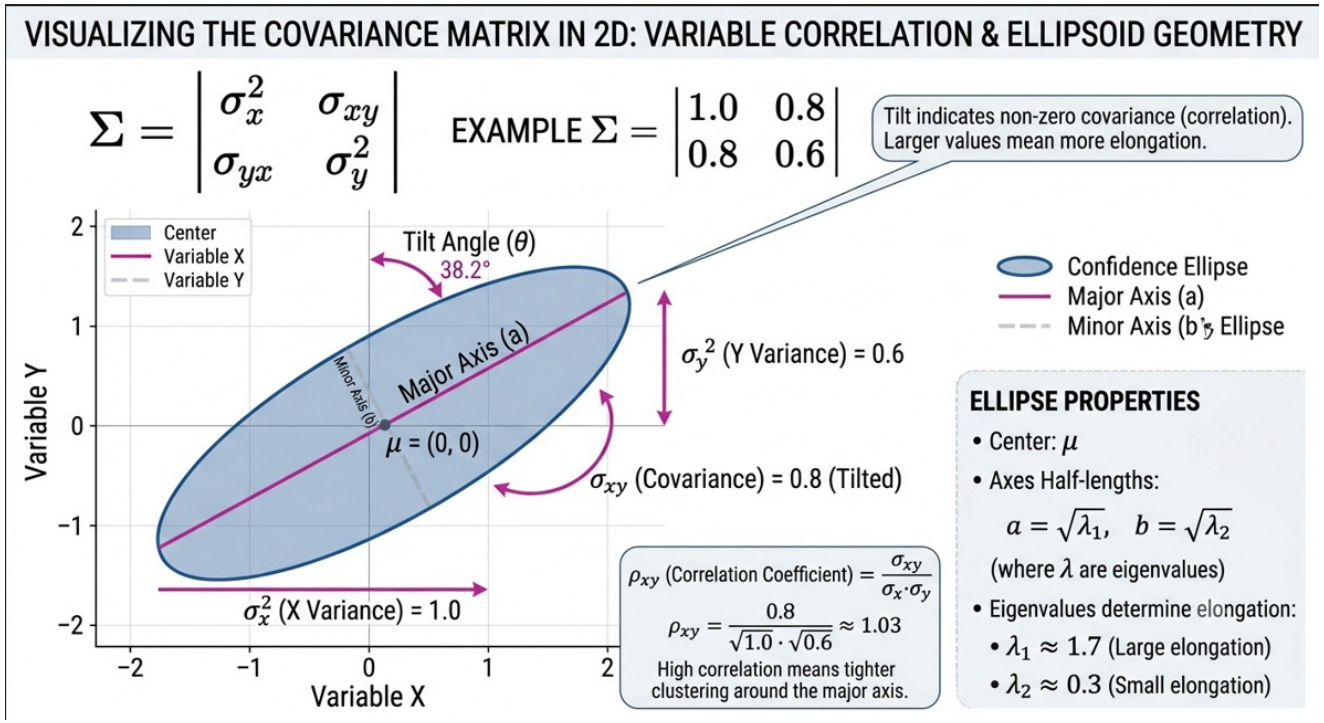
$$a = \sqrt{\lambda_1}, \quad b = \sqrt{\lambda_2}$$

(where λ are eigenvalues)

- Eigenvalues determine elongation:
  - $\lambda_1 \approx 1.7$ (Large elongation)
  - $\lambda_2 \approx 0.3$ (Small elongation)

Demonstration of Variance-Covariance Principles

- Maiti J, Multivariate Statistical Modeling in Engineering and Management, CRC Press, 2023.
- Johnson, R. A., Wishern, D. W., Applied Multivariate Statistical Analysis, sixth ed., Pearson, 2024.



1. Positive Covariance

Example: Ice cream sales versus daily temperature. As the temperature rises, ice cream sales tend to increase.

Data Points

Let X = Temperature (in °C) and Y = Ice Cream Sales (in USD).

Day	Temperature (X)	Ice Cream Sales (Y)
1	20	200
2	25	300
3	30	450

...

Calculation & Figure

- Mean of X ($\bar{X}$) = 25
- Mean of Y ($\bar{Y}$) = 316.67

2. Negative Covariance

Example: Outdoor temperature versus heating oil consumption. As the temperature goes up, the need for heating goes down.

Data Points

Let X = Temperature (in °C) and Y = Heating Bill (in USD).

Month	Temperature (X)	Heating Bill (Y)
Jan	5	150
Mar	12	90
May	22	30

...

Calculation & Figure

- Mean of X ($\bar{X}$) = 13
- Mean of Y ($\bar{Y}$) = 90

3. Zero Covariance

Example: A person's shoe size versus their monthly coffee consumption. There is no logical or statistical relationship between these two metrics.

Data Points

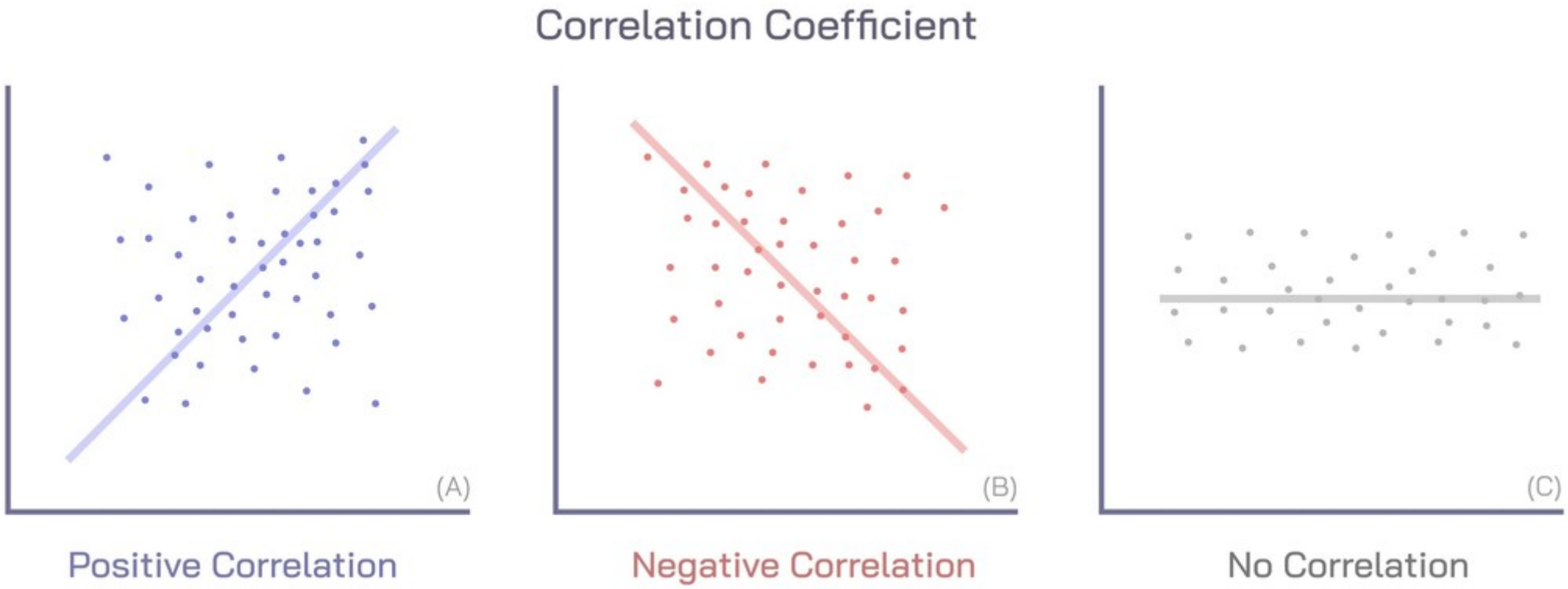
Let X = Shoe Size (US) and Y = Cups of Coffee per month.

Person	Shoe Size (X)	Cups of Coffee (Y)
A	7	40
B	9	10
C	11	40

...

Calculation & Figure

- Mean of X ($\bar{X}$) = 9
- Mean of Y ($\bar{Y}$) = 30



Source: Shuttercock

What is an S matrix? Covariance

$$s_{j,k} = \frac{1}{n-1}\sum_{i=1}^{n}\left(x_{i,j} - \bar{x}_j\right)\left(x_{i,k} - \bar{x}_k\right)$$

$$S = \begin{bmatrix} s_{11} & s_{12} & s_{13} \\ s_{12} & s_{22} & s_{23} \\ s_{13} & s_{23} & s_{33} \end{bmatrix}_{pxp} \qquad \mathbf{x_j} = \begin{bmatrix} x_{1j} \\ x_{2j} \\ x_{3j} \end{bmatrix}_{nxp} \qquad \mathbf{1} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}_{nxp}$$

In matrix form

$$s_{j,k} = \frac{1}{n-1}\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right)^{\mathbf{T}}\left(\mathbf{x_k} - \mathbf{1}\bar{x}_k\right)$$

$$s_{j,j} = \frac{1}{n-1}\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right)^{\mathbf{T}}\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right)$$

How to get the S Matrix from X Matrix?

$$s_{j,j} = \frac{1}{n-1}\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right)^T\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right) \qquad s_{j,k} = \frac{1}{n-1}\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right)^T\left(\mathbf{x_k} - \mathbf{1}\bar{x}_k\right)$$

$$\mathbf{x_j} = \begin{bmatrix} x_{1j} \\ x_{2j} \\ x_{3j} \end{bmatrix}_{nxp} \qquad \bar{\mathbf{x}} = \begin{bmatrix} \bar{x}_1 \\ \bar{x}_2 \end{bmatrix}_{2x1} = \begin{bmatrix} 8 \\ 16 \end{bmatrix}_{2x1}$$

$$\mathbf{x_1} = \begin{bmatrix} 4 \\ 8 \\ 12 \end{bmatrix}_{3x1} \qquad \mathbf{1}\bar{x}_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} 8 = \begin{bmatrix} 8 \\ 8 \\ 8 \end{bmatrix}_{2x1}$$

$$\mathbf{x_2} = \begin{bmatrix} 8 \\ 16 \\ 24 \end{bmatrix}_{3x1} \qquad \mathbf{1}\bar{x}_2 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} 16 = \begin{bmatrix} 16 \\ 16 \\ 16 \end{bmatrix}_{2x1}$$

Matlab Code

```
clc ; clear all

x1=[4 8 12]'; x2 = [8 16 24]';
X=[x1 x2];
% X =
%
% 4     8
% 8    16
% 12   24
COVX=cov(X);
% COVX =
%
% 16    32
% 32    64
val=size(X);  % [3x2]
x1bar=mean(X(:,1));   % 8
x2bar=mean(X(:,2));   % 16
onesvector=ones(val(1),1);
% onesvector =
% 1
% 1
% 1

% Verify with actual Matlab code
var11=1/(val(1)-1)*(X(:,1)-onesvector*x1bar)'*(X(:,1)-onesvector*x1bar);
var12=1/(val(1)-1)*(X(:,1)-onesvector*x1bar)'*(X(:,2)-onesvector*x2bar);
var21=1/(val(1)-1)*(X(:,2)-onesvector*x2bar)'*(X(:,1)-onesvector*x1bar);
var22=1/(val(1)-1)*(X(:,2)-onesvector*x2bar)'*(X(:,2)-onesvector*x2bar);

Cov_Matlab = [[var11, var12] ; [var21,  var22] ]

% Cov_Matlab =
% 16    32
% 32    64
CorrelationX=corr(X);
% CorrelationX =
% 1.0000    1.0000
% 1.0000    1.0000
Correlation_Matlab = [[var11/sqrt(var11*var11), var12/sqrt(var11*var22)] ; [var21/sqrt(var11*var22),  var22/sqrt(var22*var22) ]];
% Correlation_Matlab =
%
    %
    % 1     1
    % 1     1
```

$$\mathbf{S} = \frac{1}{n-1}\left[(\mathbf{X} - \bar{\mathbf{X}})^{\mathbf{T}}(\mathbf{X} - \bar{\mathbf{X}})\right]$$

$$s_{j,k} = \frac{1}{n-1}\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right)^{\mathbf{T}}\left(\mathbf{x_k} - \mathbf{1}\bar{x}_k\right)$$

$$s_{j,j} = \frac{1}{n-1}\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right)^{\mathbf{T}}\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right)$$

Sample Pearson Correlation Coefficient Given by

$$r_{XY} = \frac{Cov(X,Y)}{\sigma_X \sigma_Y}$$

$$S = \begin{bmatrix} s_{11} & s_{12} \\ s_{12} & s_{22} \end{bmatrix}_{pxp}$$

$$S = \begin{bmatrix} 16 & 32 \\ 32 & 64 \end{bmatrix}_{pxp}$$

Python Code

```
# Create column vectors
x1 = np.array([4, 8, 12]).reshape(-1, 1)
x2 = np.array([8, 16, 24]).reshape(-1, 1)

# Concatenate horizontally to form matrix X
X = np.hstack((x1, x2))
# NumPy's cov treats rows as variables by default, so we use rowvar=False to match MATLAB
COVX = np.cov(X, rowvar=False)

val = X.shape  # Returns a tuple (3, 2)
x1bar = np.mean(X[:, 0])  # 8.0
x2bar = np.mean(X[:, 1])  # 16.0
onesvector = np.ones((val[0], 1))

x1_col = X[:, 0:1]
x2_col = X[:, 1:2]

var11 = (1 / (val[0] - 1)) * ((x1_col - onesvector * x1bar).T @ (x1_col - onesvector * x1bar)).item()
var12 = (1 / (val[0] - 1)) * ((x1_col - onesvector * x1bar).T @ (x2_col - onesvector * x2bar)).item()
var21 = (1 / (val[0] - 1)) * ((x2_col - onesvector * x2bar).T @ (x1_col - onesvector * x1bar)).item()
var22 = (1 / (val[0] - 1)) * ((x2_col - onesvector * x2bar).T @ (x2_col - onesvector * x2bar)).item()

Cov_Matlab = np.array([[var11, var12], [var21, var22]])

# Calculate correlation matrix
CorrelationX = np.corrcoef(X, rowvar=False)

Correlation_Matlab = np.array([
    [var11 / np.sqrt(var11 * var11), var12 / np.sqrt(var11 * var22)],
    [var21 / np.sqrt(var11 * var22), var22 / np.sqrt(var22 * var22)]
])


print("X =\n", X)
print("\nCOVX =\n", COVX)
print("\nCorrelationX =\n", CorrelationX)
```

```
X =
 [[ 4  8]
 [ 8 16]
 [12 24]]

COVX =
 [[16. 32.]
 [32. 64.]]

CorrelationX =
 [[1. 1.]
 [1. 1.]]
```

Sample Pearson Correlation Coefficient Given by

$$r_{XY} = \frac{Cov(X,Y)}{\sigma_X \sigma_Y}$$

$$S = \begin{bmatrix} s_{11} & s_{12} \\ s_{12} & s_{22} \end{bmatrix}_{pxp}$$

$$S = \begin{bmatrix} 16 & 32 \\ 32 & 64 \end{bmatrix}_{pxp}$$

$$\mathbf{S} = \frac{1}{n-1}[(\mathbf{X} - \bar{\mathbf{X}})^{\mathbf{T}}(\mathbf{X} - \bar{\mathbf{X}})]$$

$$s_{j,k} = \frac{1}{n-1}\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right)^{\mathbf{T}}\left(\mathbf{x_k} - \mathbf{1}\bar{x}_k\right)$$

$$s_{j,j} = \frac{1}{n-1}\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right)^{\mathbf{T}}\left(\mathbf{x_j} - \mathbf{1}\bar{x}_j\right)$$

Using a Normalized Data Set

- A few precautions are in order whenever we handle a large dataset
- The features data vector **X** is of size n x p (with n rows of observations over p-predictors)
- When each of the predictors ($\mathbf{X_1, X_2,\ldots,X_p}$) have
  - a) different measurements and hence units
  - b) vary widely in their values
- Under such circumstances, some of the predictors $\mathbf{X_j, X_k, \ldots}$ may exert more influence not by their real – life significance but by the sheer weight of its magnitude
- Necessary to use correlation matrix **R** rather than **S** under the above circumstances

Using a Normalized Data Set

- We implement the following transformations on $\mathbf{X}_{n \times p}$ to get the transformed matrix $\tilde{\mathbf{X}}$ as shown below
- Define the transformed matrix $\tilde{\mathbf{X}}$ of dimension n x p where each element of this matrix is defined as shown below (i=1,2,…,n and j=1,2,…,p)

$$\tilde{x}_{i,j} = \frac{x_{i,j} - \bar{x}_j}{\sqrt{s_{jj}}}$$

- Here, $\tilde{x}_{i,j}$ is the transformed individual data, $\bar{x}_i$ is the mean of the i^{th} column predictor's data

Normalized Matrices

Let $\tilde{\mathbf{X}}_{nxp}$ be the transformed matrix of $\mathbf{X}_{nxp}$ in such a way that each element $\mathbf{x}_{ij}$ of $\mathbf{X}_{nxp}$ is subtracted from its corresponding variable mean and divided by its corresponding standard deviation s_j .

$$\tilde{x}_{i,j} = \frac{x_{i,j} - \bar{x}_j}{\sqrt{s_{jj}}} = \frac{x_{i,j} - \bar{x}_j}{s_j}$$

If x_{ij} denote an element of $\tilde{\mathbf{X}}_{nxp}$, then

We get $\tilde{\mathbf{X}}$ as shown on RHS

$$\tilde{\mathbf{X}} = \begin{bmatrix} \tilde{x}_{11} & \tilde{x}_{12} & \tilde{x}_{13} \\ \tilde{x}_{21} & \tilde{x}_{22} & \tilde{x}_{23} \\ \tilde{x}_{31} & \tilde{x}_{32} & \tilde{x}_{33} \\ \tilde{x}_{41} & \tilde{x}_{42} & \tilde{x}_{43} \end{bmatrix}_{4x3}$$

Correlation Matrix (R) For Normalized Values

$$\mathbf{S} = \frac{1}{n-1}[(\mathbf{X} - \bar{\mathbf{X}})^{\mathbf{T}}(\mathbf{X} - \bar{\mathbf{X}})]$$

$$\mathbf{R} = \frac{1}{n-1}[\tilde{\mathbf{X}}^{\mathbf{T}}\tilde{\mathbf{X}}]$$

What is **R**?

- It is the covariance matrix for the transformed variables $\tilde{x}_{ij}$ $(i = 1,2,\ldots.,n, j = 1,2,\ldots,p)$
- It is expected that the diagonal elements are variances of the transformed variables and off-diagonal elements represent co-variances between a pair of vectors $\tilde{x}_j$ and $\tilde{x}_k$ with k ≠j.
- We may find R in two ways as given below

$$\mathbf{R} = \frac{1}{n-1}[\tilde{\mathbf{X}}^{\mathbf{T}}\tilde{\mathbf{X}}]$$

Example Problem With Uneven Data

1. Lot Area (Square Feet, ft^2) – High magnitude (thousands)
2. Age (Years) – Low magnitude (single/double digits)
3. Monthly Rent (USD, $) – Medium-high magnitude (thousands)
4. Distance to Metro (Miles) – Very low magnitude / Continuous decimals
5. Annual Energy Cost (USD, $) – Very high magnitude (tens of thousands)

Example Problem With Raw Data

Property	Lot Area (sq ft)	Age (Years)	Monthly Rent ($)	Distance to Metro (r	Annual Energy Cost ($)
Obs 1	2,500	45	3200	0.25	12400
Obs 2	12,000	3	18500	1.8	48000
Obs 3	5,800	12	6100	0.5	22100
Obs 4	1,100	80	1900	0.05	8500
Obs 5	25,000	8	35000	3.2	95000
Obs 6	8,200	25	9000	1.1	31000
Obs 7	3,400	50	4200	0.4	15600
Obs 8	17,500	2	22000	2.5	64000
Obs 9	6,000	18	7800	0.85	26500
Obs 10	14,300	33	14000	1.4	52500
mean	9,580	28	12,170	1	37,560
std	7585.629689	24.86943684	10417.40531	1.031571078	27273.8051

```
% Code for Raw Data Set Normalizing and Processing
clear all ; clc
X = [
2500        45          3200        0.25        12400
12000       3           18500       1.8         48000
5800        12          6100        0.5         22100
1100        80          1900        0.05         8500
25000       8           35000       3.2         95000
8200        25          9000        1.1         31000
3400        50          4200        0.4         15600
17500       2           22000       2.5         64000
6000        18          7800        0.85        26500
14300       33          14000       1.4         52500
];

val=size(X);
X_norm = normalize(X);
% X_norm =
% -0.9333   0.6997  -0.8611  -0.9258  -0.9225
% 0.3190   -0.9892   0.6076   0.5768   0.3828
% -0.4983  -0.6273  -0.5827  -0.6834  -0.5668
% -1.1179   2.1070  -0.9859  -1.1197  -1.0655
% 2.0328   -0.7881   2.1915   1.9339   2.1061
% -0.1819  -0.1045  -0.3043  -0.1018  -0.2405
% -0.8147   0.9007  -0.7651  -0.7804  -0.8052
% 1.0441   -1.0294   0.9436   1.2554   0.9694
% -0.4719  -0.3860  -0.4195  -0.3441  -0.4055
% 0.6222    0.2171   0.1757   0.1890   0.5478
S=cov(X_norm);
% Variance-Covariance Matrix is
% 1.0000  -0.6762   0.9796   0.9796   0.9980
% -0.6762   1.0000  -0.6745  -0.7204  -0.6662
% 0.9796  -0.6745   1.0000   0.9865   0.9882
% 0.9796  -0.7204   0.9865   1.0000   0.9823
% 0.9980  -0.6662   0.9882   0.9823   1.0000

R=(1/(val(1)-1))*(X_norm'*X_norm);
% Correlation Matrix is
%  1.0000  -0.6762   0.9796   0.9796   0.9980
% -0.6762   1.0000  -0.6745  -0.7204  -0.6662
%  0.9796  -0.6745   1.0000   0.9865   0.9882
%  0.9796  -0.7204   0.9865   1.0000   0.9823
%  0.9980  -0.6662   0.9882   0.9823   1.0000
```

Covariance of Normalized Data: Using Built-In Code

Normalized Matrix

Property	Lot Area (sq ft)	Age (Years)	Monthly Rent ($)	Distance to Metro (r	Annual Energy Cost ($)
Obs 1	-0.9333	0.6997	-0.8611	-0.9258	-0.9225
Obs 2	0.3190	-0.9892	0.6076	0.5768	0.3828
Obs 3	-0.4983	-0.6273	-0.5827	-0.6834	-0.5668
Obs 4	-1.1179	2.1070	-0.9859	-1.1197	-1.0655
Obs 5	2.0328	-0.7881	2.1915	1.9339	2.1061
Obs 6	-0.1819	-0.1045	-0.3043	-0.1018	-0.2405
Obs 7	-0.8147	0.9007	-0.7651	-0.7804	-0.8052
Obs 8	1.0441	-1.0294	0.9436	1.2554	0.9694
Obs 9	-0.4719	-0.3860	-0.4195	-0.3441	-0.4055
Obs 10	0.6222	0.2171	0.1757	0.1890	0.5478

Variance Covariance Matrix

	Column 1	Column 2	Column 3	Column 4	Column 5
Column 1	1.0000	-0.6762	0.9796	0.9796	0.9980
Column 2	-0.6762	1.0000	-0.6745	-0.7204	-0.6662
Column 3	0.9796	-0.6745	1.0000	0.9865	0.9882
Column 4	0.9796	-0.7204	0.9865	1.0000	0.9823
Column 5	0.9980	-0.6662	0.9882	0.9823	1.0000

Correlation Matrix

```
% Correlation Matrix is
%  1.0000  -0.6762   0.9796   0.9796   0.9980
% -0.6762   1.0000  -0.6745  -0.7204  -0.6662
%  0.9796  -0.6745   1.0000   0.9865   0.9882
%  0.9796  -0.7204   0.9865   1.0000   0.9823
%  0.9980  -0.6662   0.9882   0.9823   1.0000
```

$$r_{XY} = \frac{Cov(X,Y)}{\sigma_X \sigma_Y}$$

Covariance of Normalized Data

```
% Verify with actual Matlab code
for i=1:val(2)
   for j=1:val(2)

var(i,j)=(1/(val(1)-1))*(X_norm(:,i)-
onesvector*Mean_Matrix(i))'*(X_norm(:,j)-
onesvector*Mean_Matrix(j));
   end
end

for i=1:val(2)
   for j=1:val(2)


      R_Norm(i,j) = var(i,j)/sqrt(var(i,i)*var(j,j));

   end
end
```

```
var =

    1.0000   -0.6762    0.9796    0.9796    0.9980
   -0.6762    1.0000   -0.6745   -0.7204   -0.6662
    0.9796   -0.6745    1.0000    0.9865    0.9882
    0.9796   -0.7204    0.9865    1.0000    0.9823
    0.9980   -0.6662    0.9882    0.9823    1.0000
'
```

```
R_Norm =

    1.0000   -0.6762    0.9796    0.9796    0.9980
   -0.6762    1.0000   -0.6745   -0.7204   -0.6662
    0.9796   -0.6745    1.0000    0.9865    0.9882
    0.9796   -0.7204    0.9865    1.0000    0.9823
    0.9980   -0.6662    0.9882    0.9823    1.0000
```

```
import numpy as np
np.set_printoptions(precision=2)

X = np.array([
   [2500,  45,  3200,  0.25, 12400],
   [12000,  3, 18500,  1.8,  48000],
   [5800,  12,  6100,  0.5,  22100],
   [1100,  80,  1900,  0.05,  8500],
   [25000,  8, 35000,  3.2,  95000],
   [8200,  25,  9000,  1.1,  31000],
   [3400,  50,  4200,  0.4,  15600],
   [17500,  2, 22000,  2.5,  64000],
   [6000,  18,  7800,  0.85, 26500],
   [14300, 33, 14000,  1.4,  52500]
])

val = X.shape  # val[0] is the number of rows (10), val[1] is the number of columns
(5)
# Normalize data (Z-score normalization)
# ddof=1 forces sample standard deviation (N-1) instead of population standard
deviation
X_norm = (X - np.mean(X, axis=0)) / np.std(X, axis=0, ddof=1)


# Using np.round to display to 4 decimal places similar to MATLAB's default view
print("X_norm =\n", np.round(X_norm, 4))


# Variance-Covariance Matrix
# rowvar=False treats each column as an independent variable
S = np.cov(X_norm, rowvar=False)


print("\nVariance-Covariance Matrix is\n", np.round(S, 4))


# To confirm above results by own code
# (1/(val(1)-1))*(X_norm'*X_norm) in MATLAB becomes:
# Note: val(1) in MATLAB (rows) corresponds to val[0] in Python
R = (1 / (val[0] - 1)) * (X_norm.T @ X_norm)


print("\nCorrelation Matrix is\n", np.round(R, 4))
```

Covariance of Normalized Data: Python

```
X_norm =
 [[-0.93  0.7  -0.86 -0.93 -0.92]
 [ 0.32 -0.99  0.61  0.58  0.38]
 [-0.5  -0.63 -0.58 -0.68 -0.57]
 [-1.12  2.11 -0.99 -1.12 -1.07]
 [ 2.03 -0.79  2.19  1.93  2.11]
 [-0.18 -0.1  -0.3  -0.1  -0.24]
 [-0.81  0.9  -0.77 -0.78 -0.81]
 [ 1.04 -1.03  0.94  1.26  0.97]
 [-0.47 -0.39 -0.42 -0.34 -0.41]
 [ 0.62  0.22  0.18  0.19  0.55]]
```

```
Variance-Covariance Matrix is
 [[ 1.   -0.68  0.98  0.98  1.  ]
 [-0.68  1.   -0.67 -0.72 -0.67]
 [ 0.98 -0.67  1.    0.99  0.99]
 [ 0.98 -0.72  0.99  1.    0.98]
 [ 1.   -0.67  0.99  0.98  1.  ]]

Correlation Matrix is
 [[ 1.   -0.68  0.98  0.98  1.  ]
 [-0.68  1.   -0.67 -0.72 -0.67]
 [ 0.98 -0.67  1.    0.99  0.99]
 [ 0.98 -0.72  0.99  1.    0.98]
 [ 1.   -0.67  0.99  0.98  1.  ]]
```

Code links

Python and Matlab code Links